Technical Documentation - RPG Party Manager

1. Introduction

RPG Party Manager is an Android application developed in Java that allows users to manage RPG-style characters and their statistics. The application demonstrates several Android development concepts including Activities, RecyclerView, Room Database, Fragments, Intents, and BroadcastReceivers.

The application enables users to:

- Create characters
- Edit characters
- Delete characters
- Modify HP and Mana values
- Store data permanently using Room Database
- Simulate RPG dice rolls with critical hit/failure detection

The project was designed to demonstrate Android architecture principles and persistent local storage.

2. Application Architecture

The application follows a layered architecture separating the user interface, business logic, and data persistence.

Architecture Overview

User Interface Layer
↓
Activity / Fragment Layer
↓
RecyclerView Adapter Layer
↓
Room Database Layer
↓
SQLite Storage

This structure improves:

- maintainability
- readability
- scalability
- separation of concerns

3. Main Components

3.1 Activities

Activities represent the main screens of the application.

MainActivity

Purpose:

- Entry point of the application
- Navigation hub

Responsibilities:

- Open Character List screen
- Open Dice Roller screen

CharacterListActivity

Purpose:

- Display all characters using RecyclerView

Responsibilities:

- Load data from Room Database
- Display character list
- Delete characters
- Open character detail screen
- Launch character creation screen

RecyclerView was selected because it efficiently handles dynamic lists and large datasets.

CharacterDetailActivity

Purpose:

- Display detailed information for a selected character

Responsibilities:

- Display HP and Mana
- Increase/decrease stats
- Save updates to Room Database
- Open Edit screen
- Share character data

This activity uses a Fragment to display reusable stat information.

CreateCharacterActivity

Purpose:

- Create and save new characters

Responsibilities:

- Collect user input
- Insert character into Room Database

EditCharacterActivity

Purpose:

- Modify existing character data

Responsibilities:

- Load existing character data
- Update Room Database entries
- Save modifications permanently

DiceRollActivity

Purpose:

- Simulate RPG dice rolling

Responsibilities:

- Generate random D20 values
- Trigger critical hit/failure events
- Send broadcasts to DiceReceiver

4. RecyclerView Architecture

RecyclerView is used to efficiently display character data.

Components Used

RecyclerView

Displays the scrollable list of characters.

CharacterAdapter

Connects character data to RecyclerView item layouts.

Responsibilities:

- Inflate item views
- Bind character data
- Handle click actions
- Handle delete button actions

ViewHolder Pattern

Used to improve performance by caching UI references.

Advantages:

- Better scrolling performance
- Reduced memory usage
- Efficient view recycling

5. Room Database Architecture

The application uses Room Persistence Library for local data storage.

Room acts as an abstraction layer over SQLite.

Advantages:

- Simplified database management
- Safer queries
- Easier CRUD operations
- Persistent storage between app launches

5.1 CharacterEntity

CharacterEntity represents the database table.

Fields

FIELD	TYPE	DESCRIPTION
ID	int	Unique primary key
NAME	String	Character name
ROLE	String	Character class
HP	int	Health points
MANA	int	Mana points

The id field uses auto-generation to ensure uniqueness.

5.2 CharacterDao

DAO stands for Data Access Object.

Responsibilities:

- Insert characters
- Update characters
- Delete characters
- Query characters

Main methods:

insert()
update()
delete()
getAll()
getById()

5.3 AppDatabase

Purpose:

- Main database instance

The database uses the Singleton pattern.

Advantages:

- Prevents duplicate database instances
- Reduces memory usage
- Centralizes database access

6. Fragment Usage

StatsFragment

Purpose:

- Display HP and Mana information

Why Fragment was used:

- reusable UI component
- modular architecture
- cleaner activity structure

The fragment is dynamically inserted into CharacterDetailActivity.

7. Advanced Android Component

BroadcastReceiver

The application uses a BroadcastReceiver named DiceReceiver.

Purpose

DiceReceiver listens for broadcasts sent by DiceRollActivity.

Why BroadcastReceiver Was Chosen

BroadcastReceiver demonstrates Android's event-driven architecture.

Advantages:

- decouples components
- allows asynchronous event handling
- demonstrates Android system communication

Dice Roll Flow

DiceRollActivity
↓
sendBroadcast(intent)
↓
DiceReceiver
↓
Critical Hit / Failure Response

Critical Event Logic

Event	Condition
Critical Hit	Roll >= 18
Critical Failure	Roll <= 2

Receiver Features

DiceReceiver:

- receives dice roll values
- triggers vibration
- displays Toast messages

8. Data Model

The application uses a simple RPG character data model.

CharacterEntity Structure

CharacterEntity
│
├── id
├── name
├── role
├── hp
└── mana

Why IDs Are Used

Originally, characters were identified by name.

Problem:

- duplicate names caused update conflicts

Solution:

- use unique IDs

Advantages:

- reliable updates
- reliable deletes
- scalable architecture

9. Permissions

The application uses the following permissions:

9.1 VIBRATE Permission

<uses-permission android:name="android.permission.VIBRATE"/>

Reason

Required for vibration feedback during:

- critical hits
- critical failures

Used inside DiceReceiver.

9.2 CAMERA Permission

<uses-permission android:name="android.permission.CAMERA"/>

Reason

The permission was initially added for future expansion features such as:

- profile picture scanning
- QR code importing
- augmented RPG tools

The current version does not actively use the camera.

10. Navigation and Intents

The application uses Intents for navigation between activities.

Example:

Intent i = new Intent(this, CharacterDetailActivity.class);

Data is passed using Intent extras:

i.putExtra("id", character.getId());

Advantages:

- simple communication
- lightweight data transfer
- standard Android navigation

11. Data Flow

The following diagram shows how data moves through the application.

User Action
↓
Activity
↓
RecyclerView / Adapter
↓
Room DAO
↓
SQLite Database
↓
Updated UI

Room Database acts as the single source of truth.

Advantages:

- prevents stale data
- ensures synchronization
- simplifies state management

12. Testing

The application was tested using:

- Android Emulator
- Manual UI testing
- Persistence verification

Test Cases

Test	Result
Create character	Passed
Edit character	Passed
Delete character	Passed
Save HP/Mana updates	Passed
Data persistence after restart	Passed
Dice rolling system	Passed
BroadcastReceiver events	Passed

13. Future Improvements

Potential future improvements include:

- Inventory system
- Equipment management
- Character avatars
- HP/Mana progress bars
- Firebase cloud synchronization
- Online multiplayer support
- Better RPG-themed UI
- Animation effects

14. Conclusion

RPG Party Manager successfully demonstrates multiple Android development concepts including:

- Activities
- RecyclerView

- Room Database
- Fragments
- BroadcastReceivers
- Intents
- CRUD operations
- Persistent local storage

The application follows a modular architecture and uses Room Database as the single source of truth to ensure reliable and scalable data management.